

```
# -*- coding: utf-8 -*-  
"""FDS A-3.ipynb
```

Automatically generated by Colab.

Original file is located at
<https://colab.research.google.com/drive/1Q3nBNgmb3lvxzb3xTZhh19VAd1dBYE4k>

```
# Student Performance EDA
```

```
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns
```

```
# Load Dataset  
df = pd.read_csv(r"/content/students.csv", sep='\t')
```

```
# Display Data  
print("First 5 Records:")  
print(df.head())
```

```
print("\nDataset Information:")  
print(df.info())
```

```
print("\nStatistical Summary:")  
print(df.describe())
```

```
# Check Missing Values  
print("\nMissing Values:")  
print(df.isnull().sum())
```

```
# Count Pass and Fail Students  
print("\nResult Count:")  
print(df["Result"].value_counts())
```

```
# Result Distribution  
sns.countplot(x="Result", data=df)  
plt.title("Student Result Distribution")  
plt.show()
```

```
# Attendance Distribution  
plt.hist(df["Attendance"])  
plt.title("Attendance Distribution")  
plt.xlabel("Attendance")  
plt.ylabel("Number of Students")  
plt.show()
```

```
# Study Hours Distribution  
plt.hist(df["Study_Hours"])  
plt.title("Study Hours Distribution")  
plt.xlabel("Study Hours")  
plt.ylabel("Number of Students")  
plt.show()
```

```
# Internal Marks Distribution  
plt.hist(df["Internal_Marks"])  
plt.title("Internal Marks Distribution")  
plt.xlabel("Internal Marks")  
plt.ylabel("Number of Students")  
plt.show()
```

```
# Attendance vs Result  
sns.boxplot(x="Result", y="Attendance", data=df)  
plt.title("Attendance vs Result")  
plt.show()
```

```
# Study Hours vs Result  
sns.boxplot(x="Result", y="Study_Hours", data=df)  
plt.title("Study Hours vs Result")  
plt.show()
```

```
# Internal Marks vs Result  
sns.boxplot(x="Result", y="Internal_Marks", data=df)  
plt.title("Internal Marks vs Result")  
plt.show()
```

```
# Convert Result to Numeric  
df["Result"] = df["Result"].map({"Pass": 1, "Fail": 0})
```

```
# Correlation Matrix  
print("\nCorrelation Matrix:")  
print(df.corr(numeric_only=True))
```

```
# Heatmap  
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")  
plt.title("Correlation Heatmap")  
plt.show()
```

```
# Conclusion  
print("\nConclusion:")  
print("1. Students with higher attendance are more likely to pass.")  
print("2. More study hours lead to better academic performance.")  
print("3. Higher internal marks increase the chances of passing.")  
print("4. Attendance, Study Hours, and Internal Marks positively affect the final result.")
```

```
# Hospital Patient Health Risk - Exploratory Data Analysis (EDA)
```

```
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns
```

```
# Load Dataset  
df = pd.read_csv(r"/content/Hospital.csv", sep='\t') # Replace with your file name
```

```
# Display Dataset  
print("First 5 Records:")  
print(df.head())
```

```
print("\nComplete Dataset:")  
print(df)
```

```
print("\nDataset Information:")  
print(df.info())
```

```
print("\nStatistical Summary:")  
print(df.describe())
```

```
# Check Missing Values  
print("\nMissing Values:")  
print(df.isnull().sum())
```

```
# Disease Status Count
```

```

print("\nDisease Status Count:")
print(df["Disease_Status"].value_counts())

# Disease Status Distribution
sns.countplot(x="Disease_Status", data=df)
plt.title("Disease Status Distribution")
plt.show()

# Age Distribution
plt.hist(df["Age"])
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Number of Patients")
plt.show()

# Sugar Level Distribution
plt.hist(df["Sugar_Level"])
plt.title("Sugar Level Distribution")
plt.xlabel("Sugar Level")
plt.ylabel("Number of Patients")
plt.show()

# Blood Pressure Distribution
plt.hist(df["BP"])
plt.title("Blood Pressure Distribution")
plt.xlabel("Blood Pressure")
plt.ylabel("Number of Patients")
plt.show()

# BMI Distribution
plt.hist(df["BMI"])
plt.title("BMI Distribution")
plt.xlabel("BMI")
plt.ylabel("Number of Patients")
plt.show()

# Age vs Disease Status
sns.boxplot(x="Disease_Status", y="Age", data=df)
plt.title("Age vs Disease Status")
plt.show()

# Sugar Level vs Disease Status
sns.boxplot(x="Disease_Status", y="Sugar_Level", data=df)
plt.title("Sugar Level vs Disease Status")
plt.show()

# Blood Pressure vs Disease Status
sns.boxplot(x="Disease_Status", y="BP", data=df)
plt.title("Blood Pressure vs Disease Status")
plt.show()

# BMI vs Disease Status
sns.boxplot(x="Disease_Status", y="BMI", data=df)
plt.title("BMI vs Disease Status")
plt.show()

# Convert Disease Status to Numeric
df["Disease_Status"] = df["Disease_Status"].map({"Yes": 1, "No": 0})

# Correlation Matrix
print("\nCorrelation Matrix:")
print(df.corr(numeric_only=True))

# Heatmap
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()

# Conclusion
print("\nConclusion:")
print("1. Patients with higher sugar levels are more likely to have the disease.")
print("2. Higher blood pressure is associated with disease.")
print("3. Patients with higher BMI have a greater risk of disease.")
print("4. Older patients tend to have a higher chance of disease.")

# Supermarket Product Sales - EDA

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from google.colab import files

# Upload CSV File
uploaded = files.upload()

# Read Uploaded File
file_name = list(uploaded.keys())[0]
df = pd.read_csv(file_name)

# Display All Records
pd.set_option('display.max_rows', None)

print("All Product Records:")
print(df)

# Dataset Information
print("\nDataset Information:")
print(df.info())

# Statistical Summary
print("\nStatistical Summary:")
print(df.describe())

# Check Missing Values
print("\nMissing Values:")
print(df.isnull().sum())

# Category Count
print("\nCategory Count:")
print(df["Category"].value_counts())

# Category Distribution
sns.countplot(x="Category", data=df)
plt.title("Product Category Distribution")
plt.show()

# Revenue Distribution
plt.hist(df["Revenue"])
plt.title("Revenue Distribution")
plt.xlabel("Revenue")
plt.ylabel("Number of Products")
plt.show()

```

```

# Quantity Sold Distribution
plt.hist(df["Quantity_Sold"])
plt.title("Quantity Sold Distribution")
plt.xlabel("Quantity Sold")
plt.ylabel("Number of Products")
plt.show()

# Price Distribution
plt.hist(df["Price"])
plt.title("Price Distribution")
plt.xlabel("Price")
plt.ylabel("Number of Products")
plt.show()

# Discount Distribution
plt.hist(df["Discount"])
plt.title("Discount Distribution")
plt.xlabel("Discount")
plt.ylabel("Number of Products")
plt.show()

# Category vs Revenue
sns.boxplot(x="Category", y="Revenue", data=df)
plt.title("Category vs Revenue")
plt.show()

# Category vs Quantity Sold
sns.boxplot(x="Category", y="Quantity_Sold", data=df)
plt.title("Category vs Quantity Sold")
plt.show()

# Price vs Revenue
sns.scatterplot(x="Price", y="Revenue", data=df)
plt.title("Price vs Revenue")
plt.show()

# Quantity Sold vs Revenue
sns.scatterplot(x="Quantity_Sold", y="Revenue", data=df)
plt.title("Quantity Sold vs Revenue")
plt.show()

# Category Wise Revenue
print("\nCategory Wise Revenue:")
print(df.groupby("Category")["Revenue"].sum())

# Correlation Matrix
print("\nCorrelation Matrix:")
print(df.corr(numeric_only=True))

# Heatmap
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()

# Top Selling Products
print("\nTop Selling Products:")
print(df.sort_values(by="Quantity_Sold", ascending=False).head())

# Highest Revenue Products
print("\nHighest Revenue Products:")
print(df.sort_values(by="Revenue", ascending=False).head())

# Conclusion
print("\nConclusion:")
print("1. Quantity sold has a strong impact on revenue.")
print("2. High-selling products contribute more revenue.")
print("3. Category-wise analysis identifies profitable categories.")
print("4. Discounts influence customer purchases.")

# Bank Loan Approval EDA

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from google.colab import files

# Upload Dataset
uploaded = files.upload()

# Read Dataset
file_name = list(uploaded.keys())[0]
df = pd.read_csv(file_name)

# Display All Records
pd.set_option('display.max_rows', None)
print("All Customer Records:")
print(df)

# Basic Information
print("\nDataset Information:")
print(df.info())

print("\nStatistical Summary:")
print(df.describe())

# Missing Values
print("\nMissing Values:")
print(df.isnull().sum())

# 1. Loan Status Distribution - Pie Chart
plt.figure(figsize=(6,6))
df["Loan_Status"].value_counts().plot.pie(
    autopct="%1.1f%%",
    startangle=90
)
plt.title("Loan Approval Status")
plt.ylabel("")
plt.show()

# 2. Income Distribution - Histogram
plt.figure(figsize=(7,5))
plt.hist(df["Income"], bins=8)
plt.title("Income Distribution")
plt.xlabel("Income")
plt.ylabel("Customers")
plt.show()

# 3. Credit Score Distribution - Histogram
plt.figure(figsize=(7,5))
plt.hist(df["Credit_Score"], bins=8)

```

```

plt.title("Credit Score Distribution")
plt.xlabel("Credit Score")
plt.ylabel("Customers")
plt.show()

# 4. Loan Amount Distribution - Histogram
plt.figure(figsize=(7,5))
plt.hist(df["Loan_Amount"], bins=8)
plt.title("Loan Amount Distribution")
plt.xlabel("Loan Amount")
plt.ylabel("Customers")
plt.show()

# 5. Income vs Credit Score - Scatter Plot
plt.figure(figsize=(7,5))
sns.scatterplot(
    x="Income",
    y="Credit_Score",
    hue="Loan_Status",
    data=df
)
plt.title("Income vs Credit Score")
plt.show()

# 6. Credit Score vs Loan Amount - Scatter Plot
plt.figure(figsize=(7,5))
sns.scatterplot(
    x="Credit_Score",
    y="Loan_Amount",
    hue="Loan_Status",
    data=df
)
plt.title("Credit Score vs Loan Amount")
plt.show()

# 7. Income vs Loan Status - Box Plot
plt.figure(figsize=(7,5))
sns.boxplot(
    x="Loan_Status",
    y="Income",
    data=df
)
plt.title("Income Based on Loan Status")
plt.show()

# 8. Credit Score vs Loan Status - Box Plot
plt.figure(figsize=(7,5))
sns.boxplot(
    x="Loan_Status",
    y="Credit_Score",
    data=df
)
plt.title("Credit Score Based on Loan Status")
plt.show()

# 9. Employment Type Analysis - Count Plot
plt.figure(figsize=(7,5))
sns.countplot(
    x="Employment_Type",
    hue="Loan_Status",
    data=df
)
plt.title("Employment Type vs Loan Status")
plt.show()

# Convert Loan Status into Numbers
df["Loan_Status"] = df["Loan_Status"].map({
    "Approved":1,
    "Rejected":0
})

# 10. Correlation Heatmap
plt.figure(figsize=(8,6))
sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
    cmap="coolwarm"
)
plt.title("Correlation Heatmap")
plt.show()

# 11. Average Values by Loan Status
print("\nAverage Income, Credit Score and Loan Amount:")
print(
    df.groupby("Loan_Status")[
        ["Income", "Credit_Score", "Loan_Amount"]
    ].mean()
)

# Conclusion
print("\nConclusion:")
print("1. Higher income customers have more loan approvals.")
print("2. Higher credit score increases loan approval chances.")
print("3. Salaried employees have higher approval rates.")
print("4. Loan amount depends on income and credit score.")

# Agricultural Crop Yield - Exploratory Data Analysis (EDA)

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from google.colab import files

# Upload Dataset
uploaded = files.upload()

# Read Dataset
file_name = list(uploaded.keys())[0]
df = pd.read_csv(file_name)

# Display All Records
pd.set_option('display.max_rows', None)

```

```

print("All Farm Records:")
print(df)

# Dataset Information
print("\nDataset Information:")
print(df.info())

# Statistical Summary
print("\nStatistical Summary:")
print(df.describe())

# Check Missing Values
print("\nMissing Values:")
print(df.isnull().sum())

# Soil Type Count
print("\nSoil Type Count:")
print(df["Soil_Type"].value_counts())

# 1. Soil Type Distribution - Pie Chart
plt.figure(figsize=(6,6))
df["Soil_Type"].value_counts().plot.pie(
    autopct="%1.1f%%",
    startangle=90
)
plt.title("Soil Type Distribution")
plt.ylabel("")
plt.show()

# 2. Rainfall Distribution - Histogram
plt.figure(figsize=(7,5))
plt.hist(df["Rainfall_mm"], bins=8)
plt.title("Rainfall Distribution")
plt.xlabel("Rainfall (mm)")
plt.ylabel("Farms")
plt.show()

# 3. Temperature Distribution - Histogram
plt.figure(figsize=(7,5))
plt.hist(df["Temperature"], bins=8)
plt.title("Temperature Distribution")
plt.xlabel("Temperature")
plt.ylabel("Farms")
plt.show()

# 4. Fertilizer Usage Distribution - Histogram
plt.figure(figsize=(7,5))
plt.hist(df["Fertilizer_kg"], bins=8)
plt.title("Fertilizer Usage Distribution")
plt.xlabel("Fertilizer (kg)")
plt.ylabel("Farms")
plt.show()

# 5. Crop Yield Distribution - Histogram
plt.figure(figsize=(7,5))
plt.hist(df["Crop_Yield_kg"], bins=8)
plt.title("Crop Yield Distribution")
plt.xlabel("Crop Yield (kg)")
plt.ylabel("Farms")
plt.show()

# 6. Rainfall vs Crop Yield - Scatter Plot
plt.figure(figsize=(7,5))
sns.scatterplot(
    x="Rainfall_mm",
    y="Crop_Yield_kg",
    hue="Soil_Type",
    data=df
)
plt.title("Rainfall vs Crop Yield")
plt.show()

# 7. Temperature vs Crop Yield - Scatter Plot
plt.figure(figsize=(7,5))
sns.scatterplot(
    x="Temperature",
    y="Crop_Yield_kg",
    hue="Soil_Type",
    data=df
)
plt.title("Temperature vs Crop Yield")
plt.show()

# 8. Fertilizer vs Crop Yield - Scatter Plot
plt.figure(figsize=(7,5))
sns.scatterplot(
    x="Fertilizer_kg",
    y="Crop_Yield_kg",
    hue="Soil_Type",
    data=df
)
plt.title("Fertilizer Usage vs Crop Yield")
plt.show()

# 9. Soil Type vs Crop Yield - Box Plot
plt.figure(figsize=(7,5))
sns.boxplot(
    x="Soil_Type",
    y="Crop_Yield_kg",
    data=df
)
plt.title("Soil Type vs Crop Yield")
plt.show()

# 10. Soil Type vs Rainfall - Box Plot
plt.figure(figsize=(7,5))
sns.boxplot(
    x="Soil_Type",
    y="Rainfall_mm",
    data=df
)
plt.title("Soil Type vs Rainfall")
plt.show()

```

```
# Convert Soil Type into Numbers
df["Soil_Type"] = df["Soil_Type"].map({
    "Loamy":1,
    "Clay":2,
    "Sandy":3
})

# 11. Correlation Matrix
print("\nCorrelation Matrix:")
print(df.corr(numeric_only=True))

# 12. Heatmap
plt.figure(figsize=(8,6))
sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
    cmap="coolwarm"
)
plt.title("Correlation Heatmap")
plt.show()

# 13. Average Crop Yield Based on Soil Type
print("\nAverage Crop Yield by Soil Type:")
print(df.groupby("Soil_Type")["Crop_Yield_kg"].mean())

# Conclusion
print("\nConclusion:")
print("1. Higher rainfall increases crop yield.")
print("2. Proper fertilizer usage improves crop production.")
print("3. Loamy soil gives higher crop yield compared to other soil types.")
print("4. Temperature affects crop production.")
print("5. Rainfall, fertilizer, and soil type are important factors influencing yield.")
```